

PACKET CAPTURE: WIRELESS PACKET

Byeong-hee Roh

Dept. of Software and Computer Engineering

Ajou University



변화와 융합의 중심
MMcN

멀티미디어 융합 연구실

Mobile Multimedia convergene Network Lab.
<http://mmcn.ajou.ac.kr>



Contents

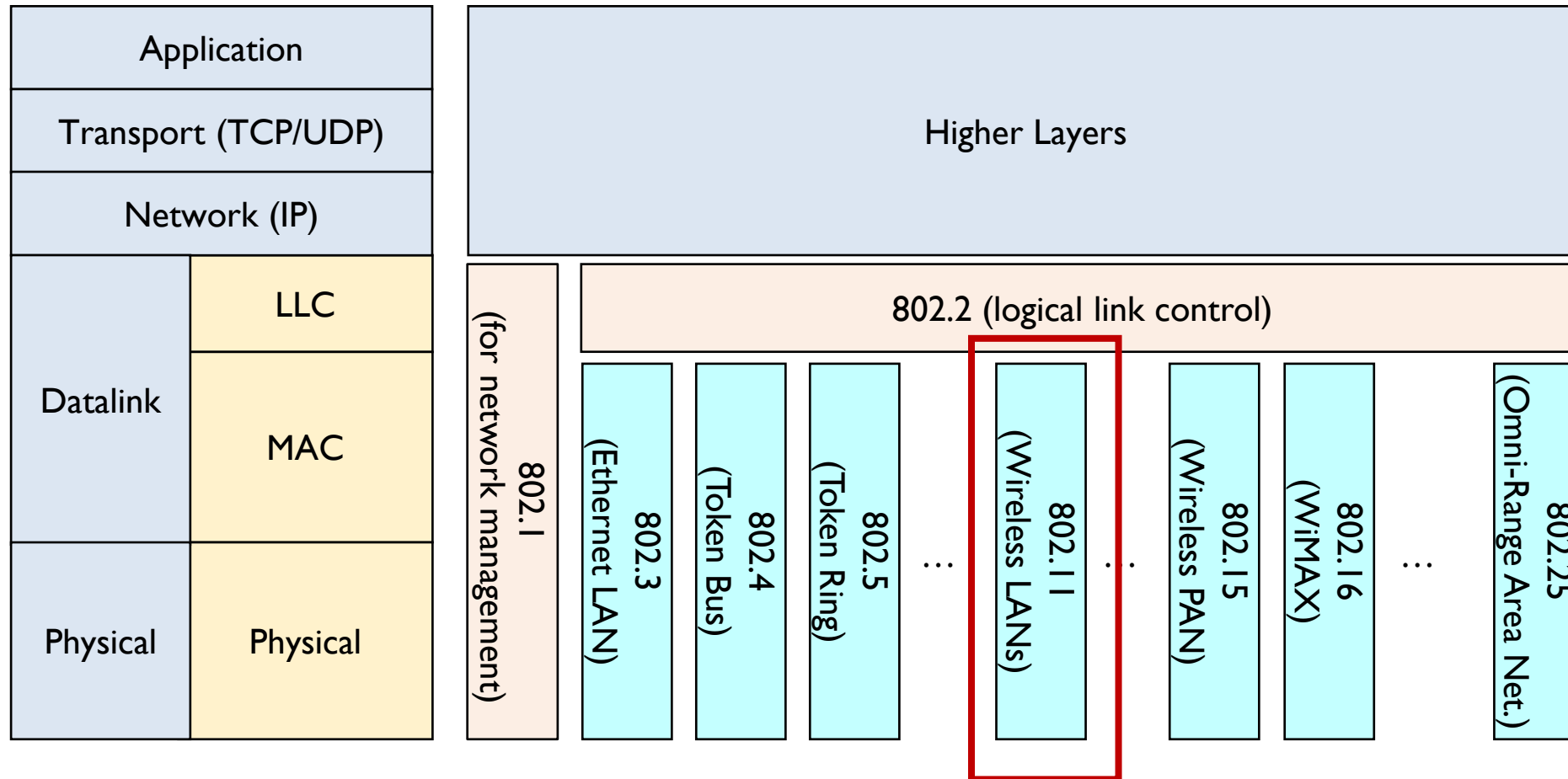
- ❖ 802.11 WLAN Frame Format
- ❖ Radiotap
- ❖ WLAN monitor mode



802.11 WLAN FRAME FORMAT

IEEE 802.11 Wireless LANs (WLANs)

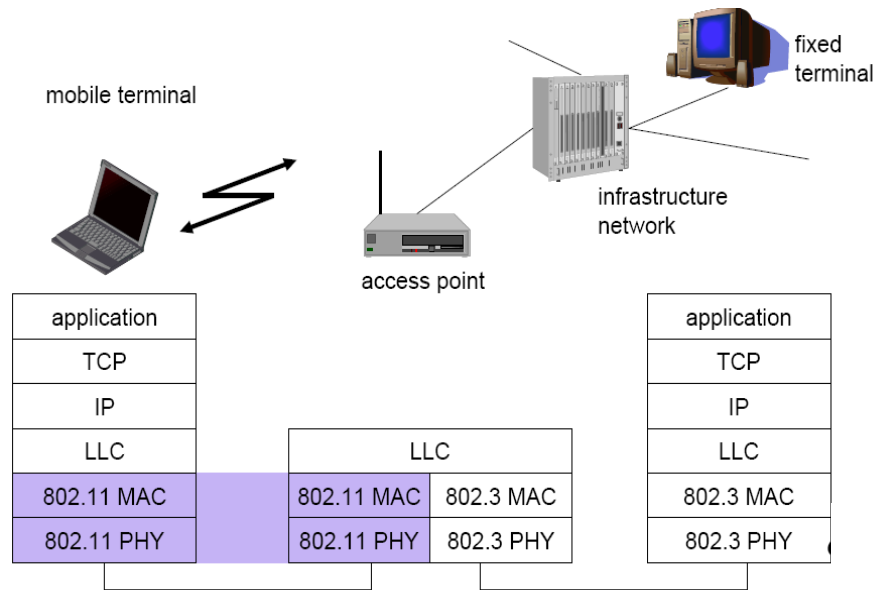
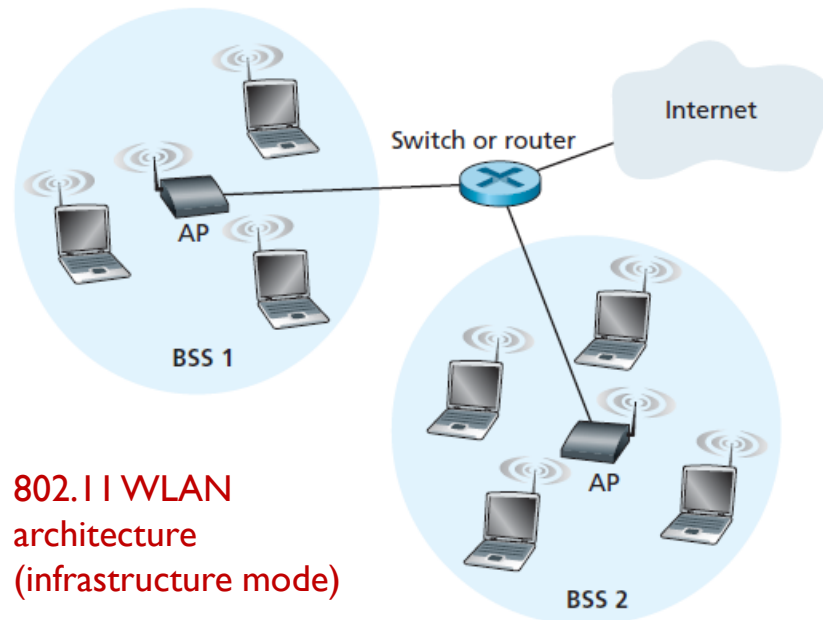
❖ IEEE 802 Standards



WLAN Architecture

❖ IEEE 802.11 WLAN architecture for infrastructure mode

- AP(access point)-based
 - WLAN hosts communicate with AP
- Basic Service Set (BSS)
 - a set of stations controlled by a single AP

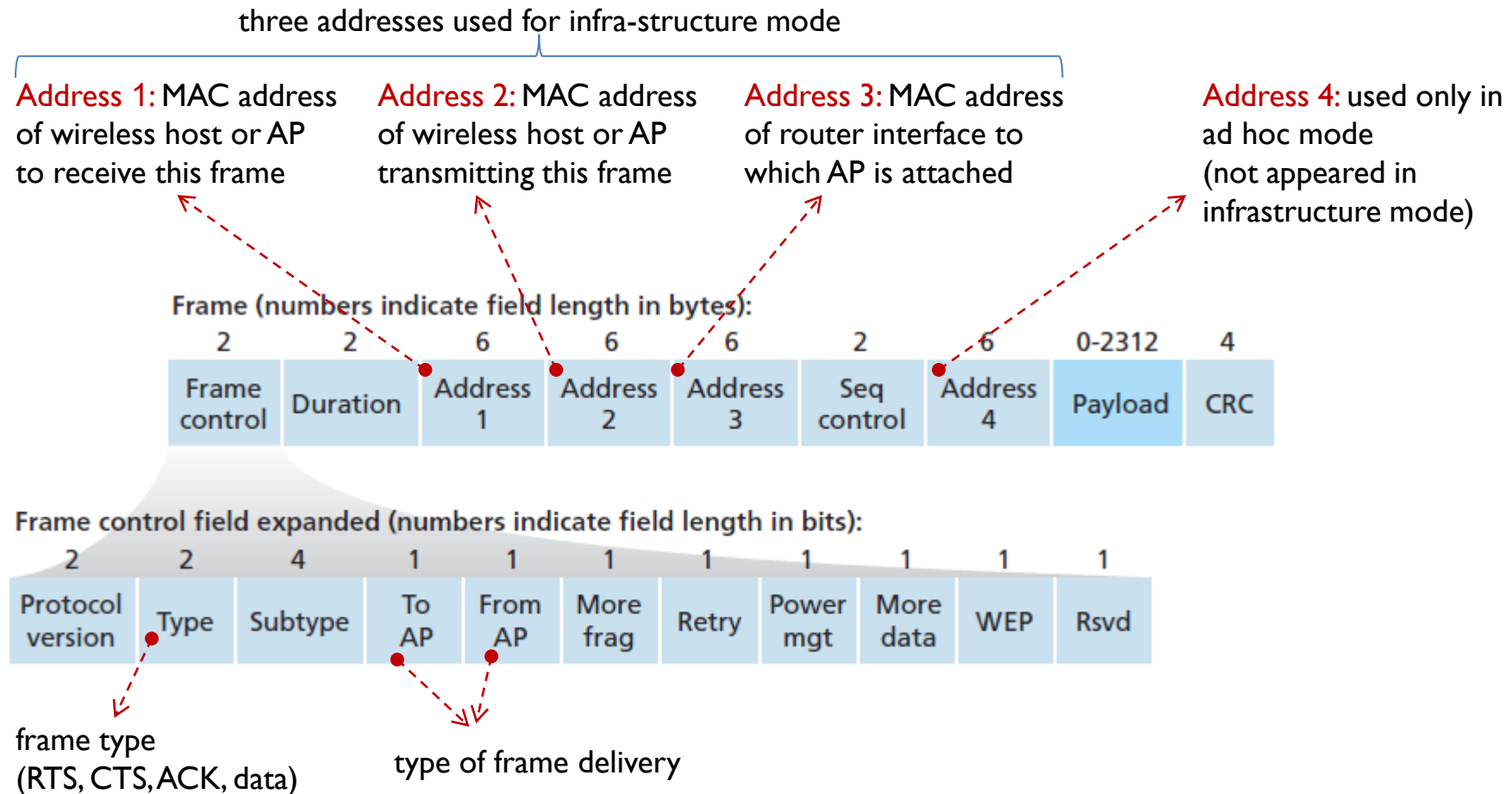


protocol stack to interconnect
802.11 WLAN with 802.3 Ethernet

IEEE 802.11 Frame Format

❖ IEEE 802.11 frame format

▪ four addresses



IEEE 802.11 Frame Format

❖ 802.11 frame control

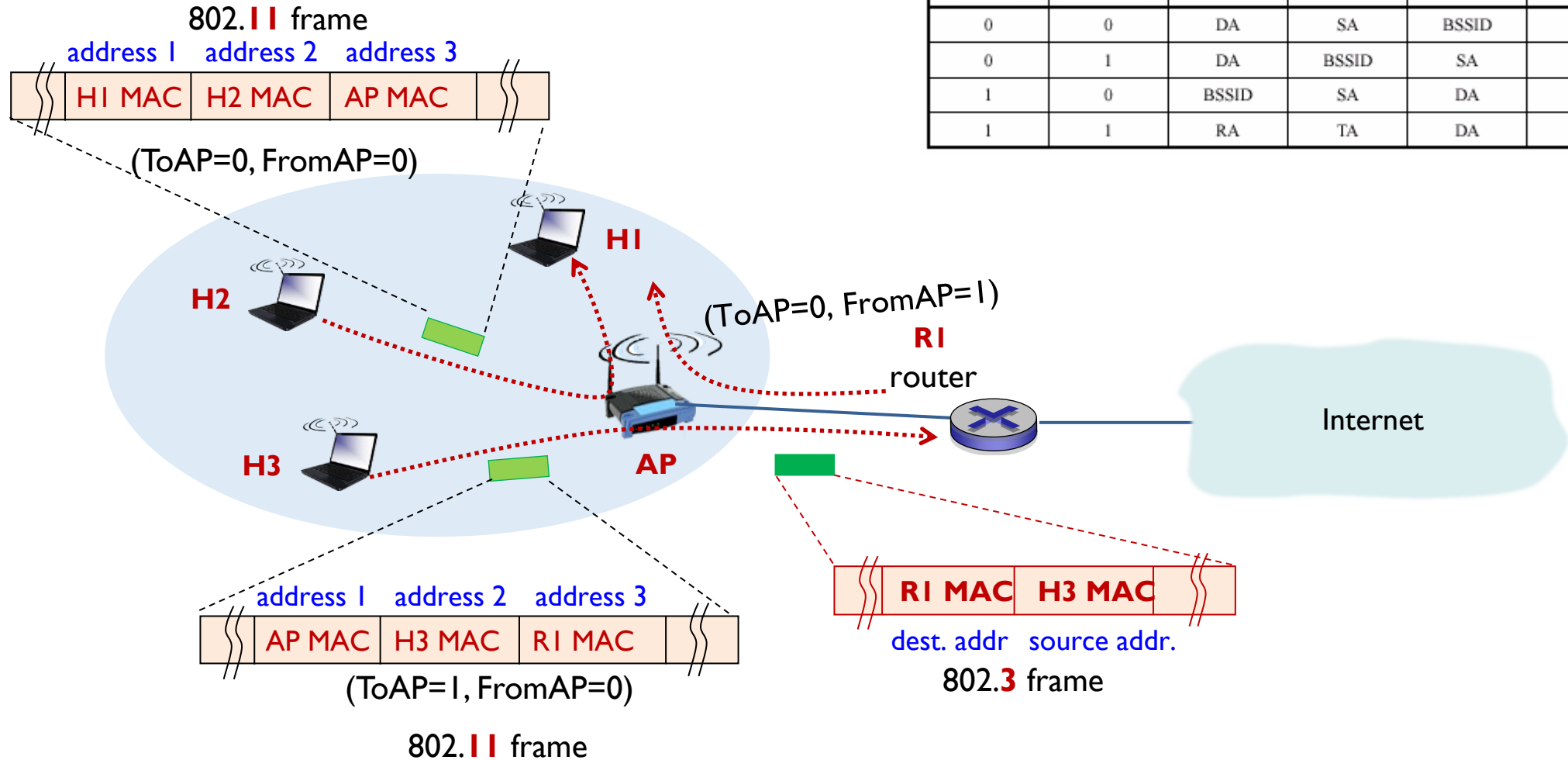
■ Type & Subtype

Type value b3 b2	Type description	Subtype value b7 b6 b5 b4	Subtype description
00	Management	0000	Association request
00	Management	0001	Association response
00	Management	0010	Reassociation request
00	Management	0011	Reassociation response
00	Management	0100	Probe request
00	Management	0101	Probe response
00	Management	0110–0111	Reserved
00	Management	1000	Beacon
00	Management	1001	Announcement traffic indication message (ATIM)
00	Management	1010	Disassociation
00	Management	1011	Authentication
00	Management	1100	Deauthentication
00	Management	1101–1111	Reserved

Type value b3 b2	Type description	Subtype value b7 b6 b5 b4	Subtype description
01	Control	0000–1001	Reserved
01	Control	1010	Power Save (PS)-Poll
01	Control	1011	Request To Send (RTS)
01	Control	1100	Clear To Send (CTS)
01	Control	1101	Acknowledgment (ACK)
01	Control	1110	Contention-Free (CF)-End
01	Control	1111	CF-End + CF-Ack
10	Data	0000	Data
10	Data	0001	Data + CF-Ack
10	Data	0010	Data + CF-Poll
10	Data	0011	Data + CF-Ack + CF-Poll
10	Data	0100	Null function (no data)
10	Data	0101	CF-Ack (no data)
10	Data	0110	CF-Poll (no data)
10	Data	0111	CF-Ack + CF-Poll (no data)
10	Data	1000–1111	Reserved
11	Reserved	0000–1111	Reserved

IEEE 802.11 Frame Format

❖ Addresses



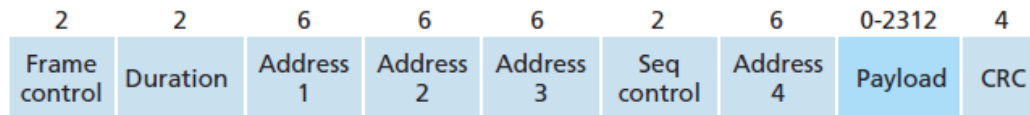
ToAP	FromAP	Address 1	Address 2	Address 3	Address 4
0	0	DA	SA	BSSID	N/A
0	1	DA	BSSID	SA	N/A
1	0	BSSID	SA	DA	N/A
1	1	RA	TA	DA	SA

IEEE 802.11 Frame Format

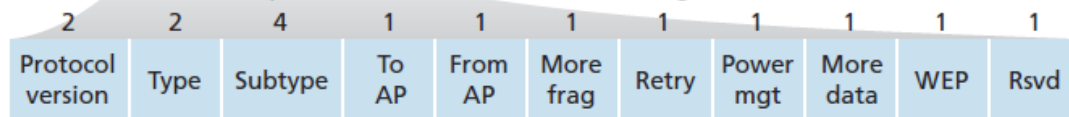
❖ Bit-order in frame control field

▪ Data frame

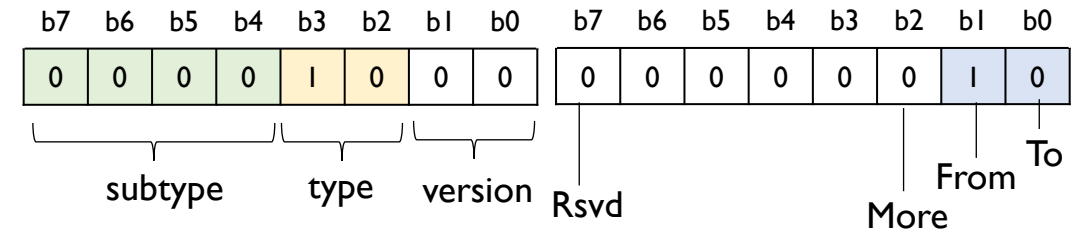
Frame (numbers indicate field length in bytes):



Frame control field expanded (numbers indicate field length in bits):



Type value b3 b2	Type description	Subtype value b7 b6 b5 b4	Subtype description
10	Data	0000	Data



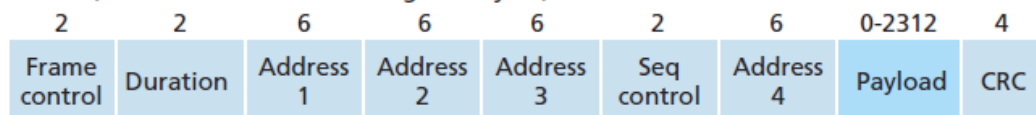
```
Type/Subtype: Data (0x0020)
▼ Frame Control Field: 0x0802
.... ..00 = Version: 0
.... 10.. = Type: Data frame (2)
0000 .... = Subtype: 0
▼ Flags: 0x02
.... ..10 = DS status: Frame from DS to a STA via AP(To DS: 0 From DS: 1) (0x2)
.... .0.. = More Fragments: This is the last fragment
.... 0... = Retry: Frame is not being retransmitted
...0 .... = PWR MGT: STA will stay up
..0. .... = More Data: No data buffered
.0.. .... = Protected flag: Data is not protected
0... .... = +HTC/Order flag: Not strictly ordered
```

IEEE 802.11 Frame Format

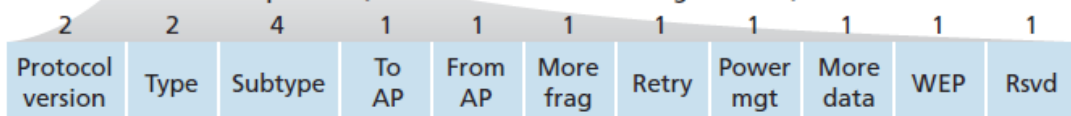
❖ Bit-order in frame control field

▪ Beacon frame

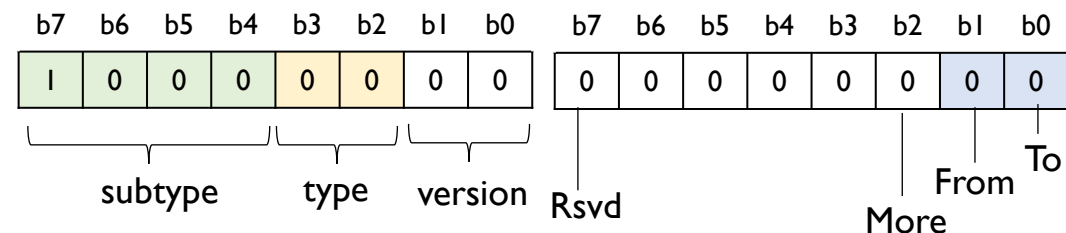
Frame (numbers indicate field length in bytes):



Frame control field expanded (numbers indicate field length in bits):



Type value b3 b2	Type description	Subtype value b7 b6 b5 b4	Subtype description
00	Management	1000	Beacon

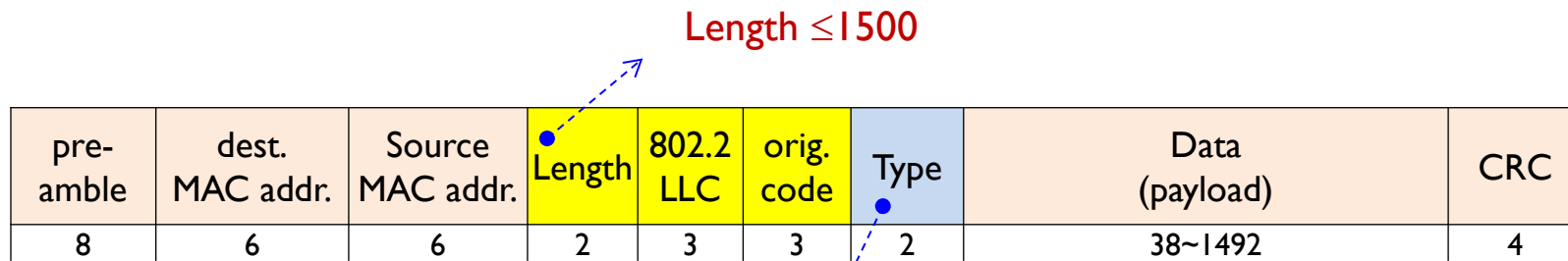


```
Type/Subtype: Beacon frame (0x0008)
✓ Frame Control Field: 0x8000
  .... ..00 = Version: 0
  .... 00.. = Type: Management frame (0)
  1000 .... = Subtype: 8
✓ Flags: 0x00
  .... ..00 = DS status: Not leaving DS or network is operating in AD-HOC mode (To DS: 0 From DS: 0) (0x0)
  .... .0.. = More Fragments: This is the last fragment
  .... 0... = Retry: Frame is not being retransmitted
  ...0 .... = PWR MGT: STA will stay up
  ..0. .... = More Data: No data buffered
  .0.. .... = Protected flag: Data is not protected
  0... .... = +HTC/Order flag: Not strictly ordered
```

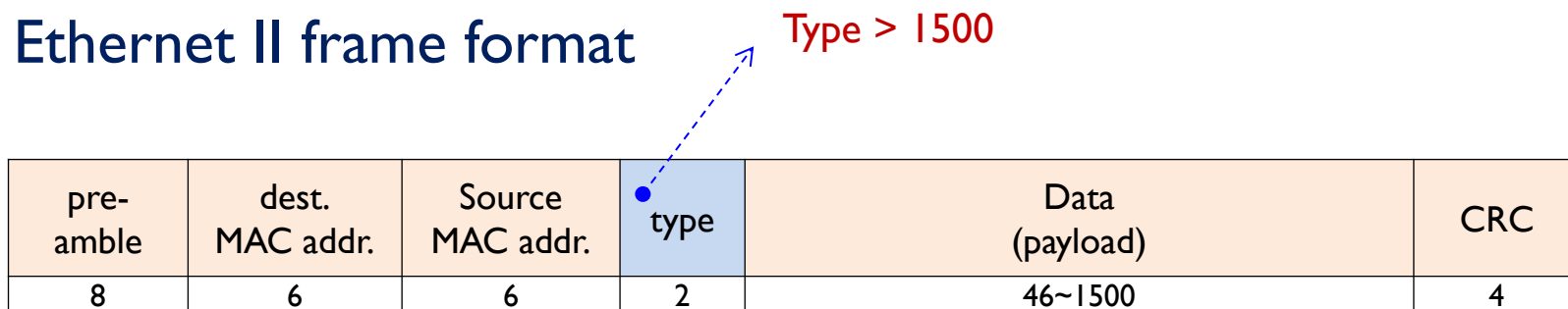
IEEE 802.11 Frame Format

❖ Ethernet Frames

■ IEEE 802.3 Ethernet frame format

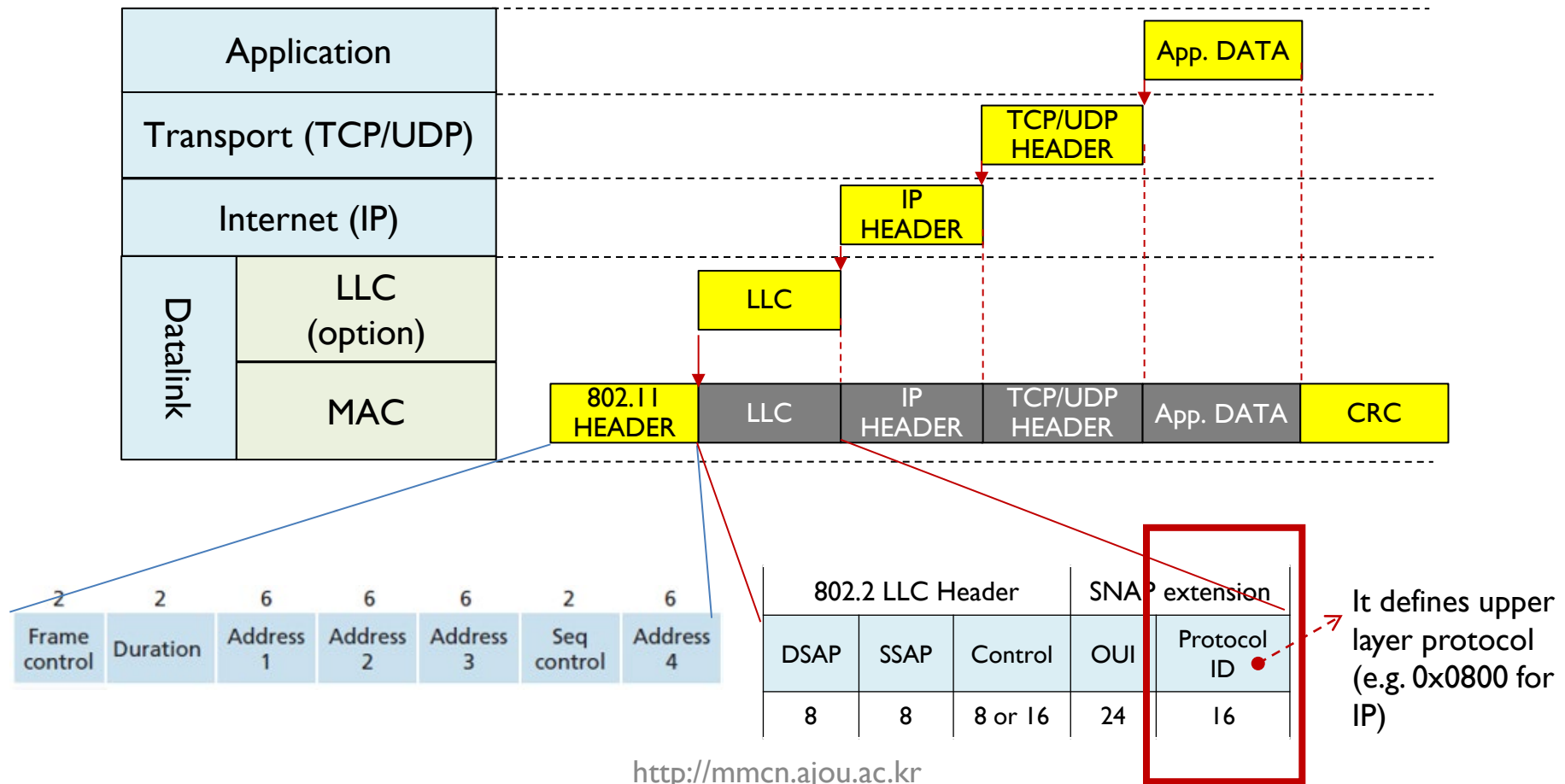


■ Ethernet II frame format



IEEE 802.11 Frame Format

- ❖ Where is type field in 802.11 frame ?
 - When 802.11 frame type is Data, an LLC header is included





IEEE 802.11 Beacon Frame

❖ Beacon frame

- a type of management frame in IEEE 802.11 WLANs
 - containing information about the network
 - transmitting periodically
 - to announce the presence of a wireless LAN and
 - to provide a timing signal to synchronize communications with the devices using the network (the members of a service set)
- In an infrastructure basic service set (BSS)
 - beacon frames are transmitted by the access point (AP)
- In ad hoc (IBSS) networks
 - beacon generation is distributed among the stations

IEEE 802.11 Beacon Frame

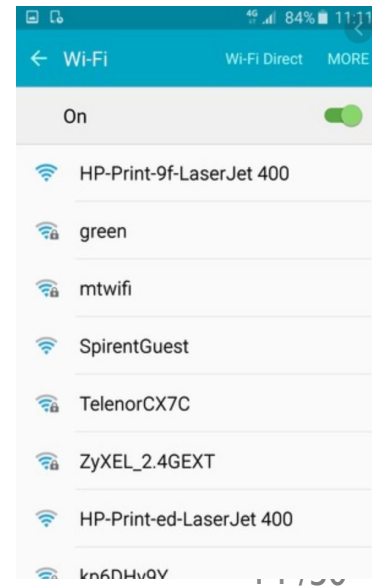
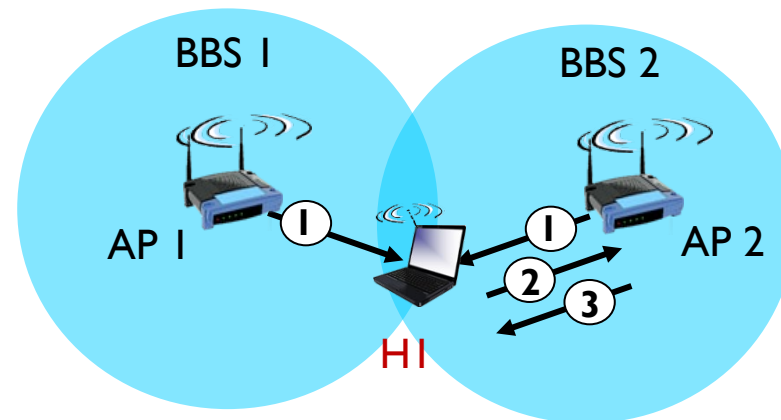
❖ Hosts must associate with an AP

- two processes for WLAN association – passive and active

Type value b3 b2	Type description	Subtype value b7 b6 b5 b4	Subtype description
00	Management	0000	Association request
00	Management	0001	Association response
00	Management	0010	Reassociation request
00	Management	0011	Reassociation response
00	Management	0100	Probe request
00	Management	0101	Probe response
00	Management	0110–0111	Reserved
00	Management	1000	Beacon
00	Management	1001	Announcement traffic indication message (ATIM)
00	Management	1010	Disassociation
00	Management	1011	Authentication
00	Management	1100	Deauthentication
00	Management	1101–1111	Reserved

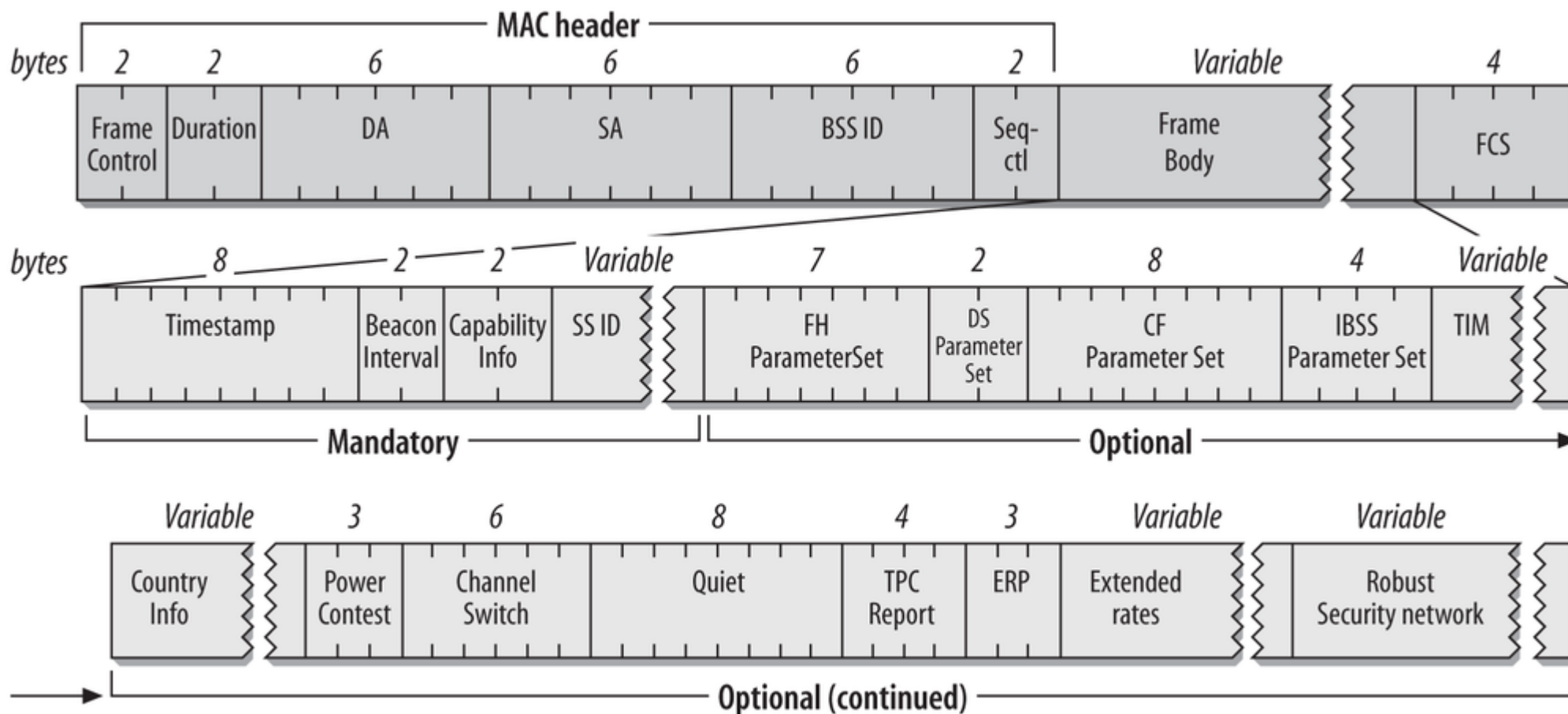
● passive scanning:

- ① Beacon frames sent from APs
- ② Association Request frame sent: HI to selected AP
- ③ Association Response frame sent from selected AP to HI



IEEE 802.11 Beacon Frame

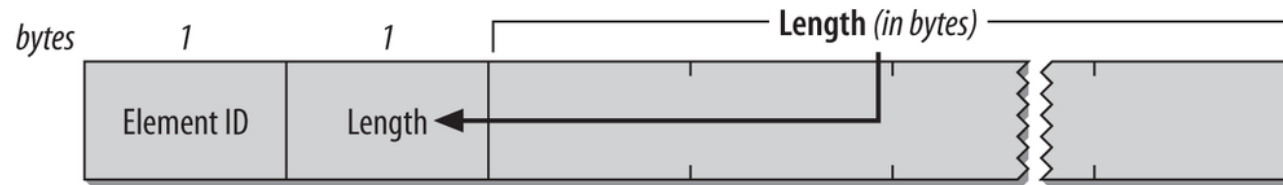
❖ Beacon Frame Format



IEEE 802.11 Beacon Frame

❖ Management Frame Information Elements

■ General



Element ID	Name
0	Service Set Identity (SSID)
1	Supported Rates
2	FH Parameter Set
3	DS Parameter Set
4	CF Parameter Set
5	Traffic Indication Map (TIM)
6	IBSS Parameter Set
7 (802.11d)	Country
8 (802.11d)	Hopping Pattern Parameters
9 (802.11d)	Hopping Pattern Table
10 (802.11d)	Request
11-15	Reserved; unused
16	Challenge text
17-31	Reserved
32 (802.11h)	Power Constraint

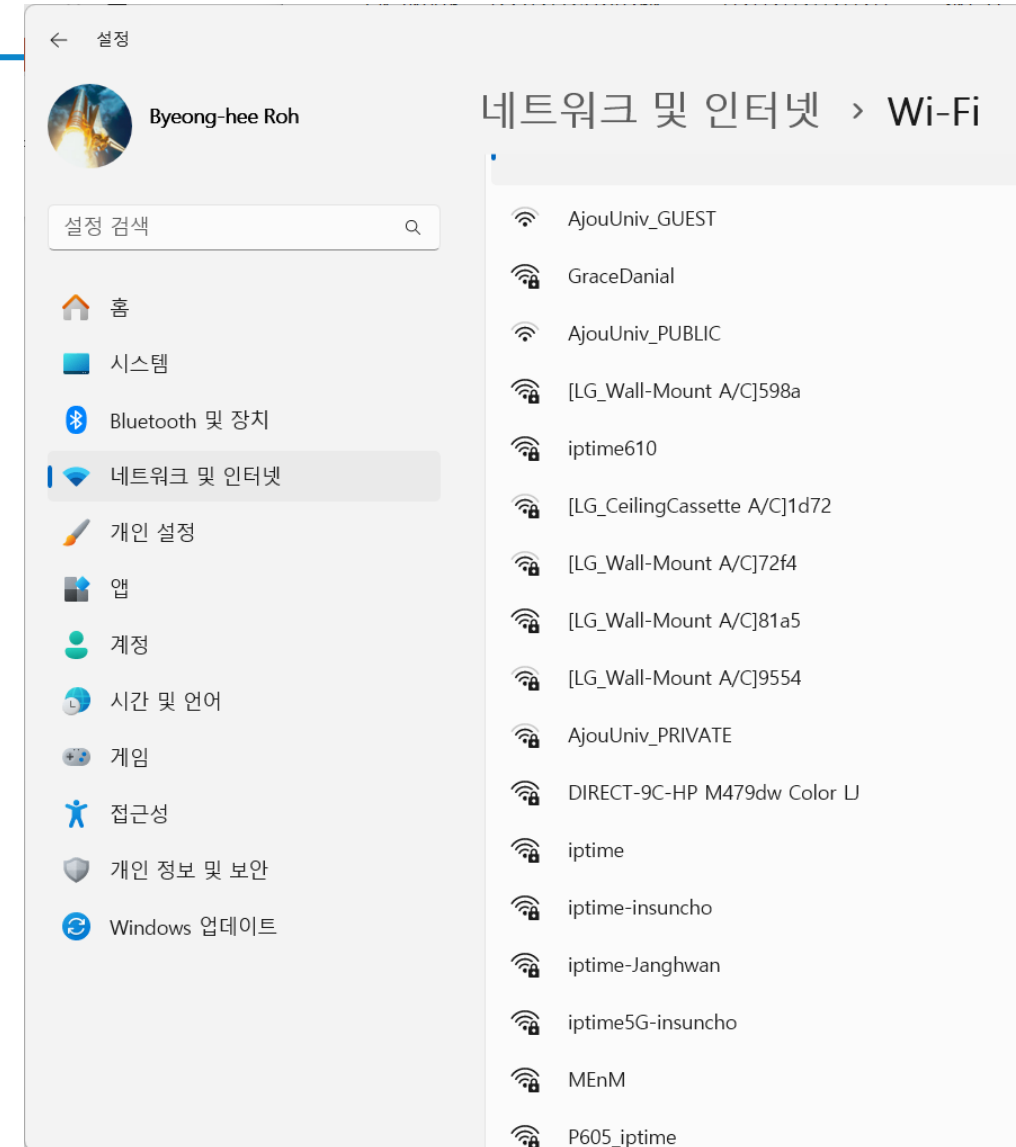
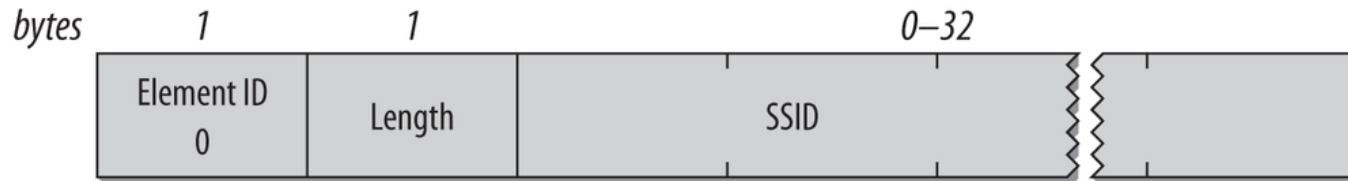
Element ID	Name
33 (802.11h)	Power Capability
34 (802.11h)	Transmit Power Control (TPC) Request
35 (802.11h)	TPC Report
36 (802.11h)	Supported Channels
37 (802.11h)	Channel Switch Announcement
38 (802.11h)	Measurement Request
39 (802.11h)	Measurement Report
40 (802.11h)	Quiet
41 (802.11h)	IBSS DFS
42 (802.11g)	ERP information
43-49	Reserved
48 (802.11i)	Robust Security Network
50 (802.11g)	Extended Supported Rates
32-255	Reserved; unused
221	Wi-Fi Protected Access

IEEE 802.11 Beacon Frame

❖ Management Frame Information Elements

■ Service Set Identity (SSID)

- is a unique identifier (or name) given to a WLAN
- allows devices to connect to a Wi-Fi network
- differentiates between multiple Wi-Fi networks in an area so users can connect to the right one.





MONITOR MODE



Monitor Mode

- ❖ Seven modes that 802.11 wireless cards can operate in
 - Master (acting as an AP), Managed (client, also known as STA),
 - Ad hoc, Mesh, Repeater,
 - Promiscuous, and
 - Monitor mode.
- ❖ Monitor mode
 - also called RFMON (Radio Frequency MONitor) mode
 - to monitor all traffic received from the wireless network
 - It allows packets to be captured without having to associate with an access point or ad hoc network first.
 - only applies to wireless networks
- ❖ Promiscuous mode
 - used for packet sniffing
 - used on both wired and wireless networks

Monitor Mode

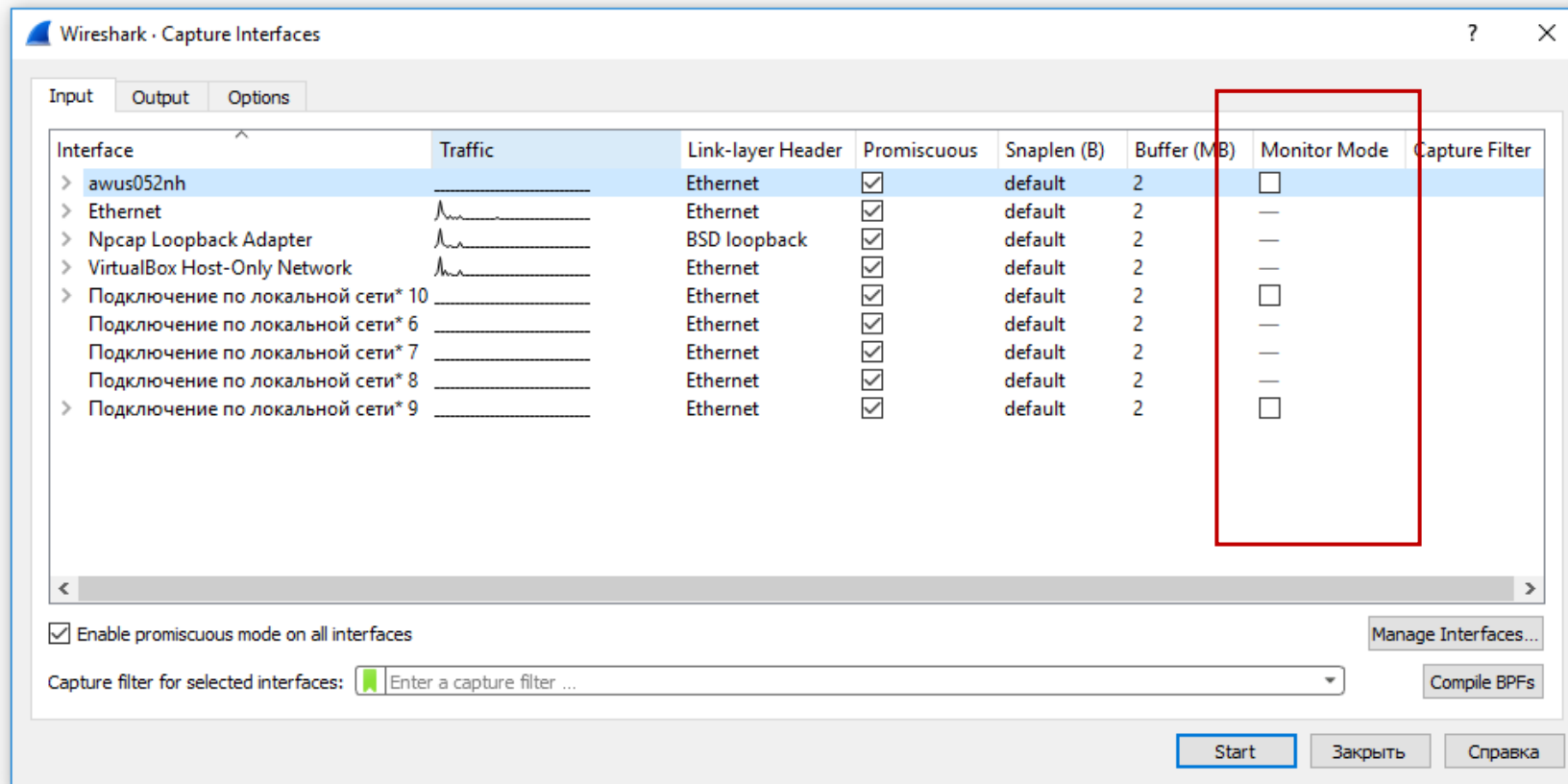
❖ Monitor and Promiscuous mode

Promiscuous	Monitor	vi	Issues to Consider
On	Off	- Fake Ethernet header prepended to packet - no Management or Control packets captured	Problems? (Disable Promiscuous Mode)
Off	Off	- Fake Ethernet header prepended to packet - no Management or Control packets captured	Need to capture traffic on the host you're interested in
Off	On	- 802.11 header - Management and Control packets captured	Need to capture traffic on the host you're interested in
On	On	- 802.11 header - Management and Control packets captured	Great. Can capture traffic on various channels and from all SSIDs

- WiFi adaptors to support monitor mode
 - https://secwiki.org/w/Npcap/WiFi_adapters

Monitor Mode

- ❖ Check whether a WiFi adaptor can support monitor mode
 - at wireshark



Monitor Mode

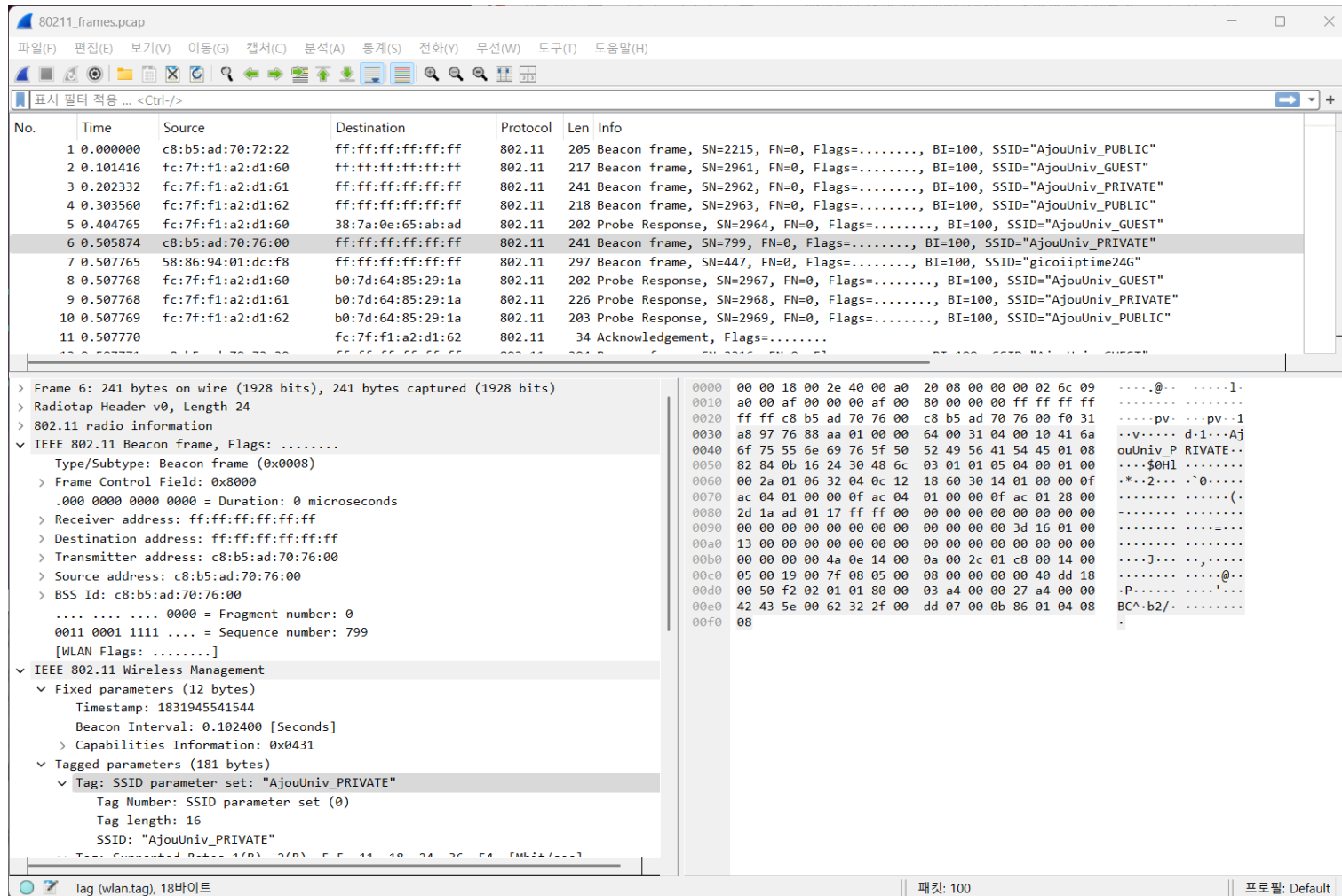
- ❖ a WiFi USB adaptor to support monitor mode

ip**TIME**
N150UA2
IEEE 802.11n 1Tx-1Rx 150Mbps Wireless USB LAN card



Monitor Mode

❖ WiFi adaptors to support monitor mode – Wireshark Capture



Monitor Mode – Functions

❖ pcap_set_rfmon()

- set monitor mode for a not-yet-activated capture handle

```
int pcap_set_rfmon ( pcap_t      *p,  
                     int         rfmon);
```

- rfmon
 - If rfmon is non-zero, monitor mode will be set
 - otherwise it will not be set.
- return value
 - returns 0 on success
 - PCAP_ERROR_ACTIVATED if called on a capture handle that has been activated.

Monitor Mode – Functions

❖ pcap_set_rfmon() for linux

▪ Example usage

```
char error_buffer[PCAP_ERRBUF_SIZE];
pcap_t *handle = pcap_create("wlan0", error_buffer);

pcap_set_rfmon(handler, 1);

pcap_set_promisc(handler, 1);          /* Capture packets that are not yours */
pcap_set_snaplen(handler, 2048);      /* Snapshot length */
pcap_set_timeout(handler, 1000);      /* Timeout in milliseconds */
pcap_activate(handle);
/* handle is ready for use with pcap_next() or pcap_loop() */
```

Set Monitor Mode in Linux

❖ Ubuntu case

- get the WLAN interface name

```
$ iwconfig
```

```
lo          no wireless extensions.
```

```
ens33       no wireless extensions.
```

```
wlan0       IEEE 802.11 Mode:Managed Frequency:2.412 GHz Tx-Power=20 dBm  
Retry short limit:7 RTS thr:off Fragment thr:off  
Power Management:off
```

- procedure to set monitor mode to the WLAN interface

```
$ sudo ifconfig wlan0 down
```

```
$ sudo iwconfig wlan0 mode monitor
```

```
$ sudo ifconfig wlan0
```

Set Monitor Mode in Linux

❖ Ubuntu case

- check the WLAN interface's monitor mode

```
$ iwconfig
```

```
lo          no wireless extensions.
```

```
ens33       no wireless extensions.
```

```
wlan0       IEEE 802.11 Mode:Monitor Frequency:2.412 GHz Tx-Power=20 dBm  
Retry short limit:7 RTS thr:off Fragment thr:off  
Power Management:off
```

WLAN Capture and Dump Example

❖ wlan_dump.c (1/3)

```
int main(int argc, char *argv[])
{
    ...

    // Retrieve the device list
    pcap_findalldevs (&alldevs, errbuf);

    // display device list
    ...

    // select device for online packet capture
    printf("Enter the interface number (1-%d):", ndNum);
    scanf("%d", &devNum);

    // Jump to the selected adapter
    for( d=alldevs, i=0; i< devNum-1 ;d=d->next, i++);

    // Open the adapter
    adhandle = pcap_open_live( d->name, 65536, 1, 1000, errbuf);
```

WLAN Capture and Dump Example

❖ wlan_dump.c (2/3)

```
// PCAP file open
dumpfile = pcap_dump_open(handle, filename);

// Activate monitor mode to the device
pcap_set_rfmon(adhandle, 1);

// display device list
pcap_dispatch(adhandle, 500, packet_handler, (u_char*)dumpfile);

pcap_dump_close(dumpfile);
pcap_close(handle);

return 0;
}
```



WLAN Capture and Dump Example

❖ wlan_dump.c (3/3)

```
void packet_handler(u_char* dumpfile, const struct pcap_pkthdr* header, const u_char* packet)
{
    // store captured packets into PCAP file
    pcap_dump(dumpfile, header, packet);

    // simple display
    printf("Captured a packet of length %d\n", header->len);
}
```

WLAN Capture and Dump Example

❖ run

```
Available devices:
1. ens33 - No description available
2. any - Pseudo-device that captures on all interfaces
3. lo - No description available
4. wlan0 - No description available
5. bluetooth-monitor - Bluetooth Linux Monitor
6. nflog - Linux netfilter log (NFLOG) interface
7. nfqueue - Linux netfilter queue (NFQUEUE) interface
8. dbus-system - D-Bus system bus
9. dbus-session - D-Bus session bus
Enter the interface number (1-9): 4
Saving 802.11 frames to 80211_frames.pcap
Captured a packet of length 205
Captured a packet of length 217
Captured a packet of length 241
Captured a packet of length 218
Captured a packet of length 202
Captured a packet of length 241
Captured a packet of length 297
Captured a packet of length 202
...
```

WLAN Capture and Dump Example



80211_frames.pcap

파일(F) 편집(E) 보기(V) 이동(G) 캡처(C) 분석(A) 통계(S) 전화(Y) 무선(W) 도구(T) 도움말(H)

표시 필터 적용 ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Len	Info
1	0.000000	c8:b5:ad:70:72:22	ff:ff:ff:ff:ff:ff	802.11	205	Beacon frame, SN=2215, FN=0, Flags=....., BI=100, SSID="AjouUniv_PUBLIC"
2	0.101416	fc:7f:f1:a2:d1:60	ff:ff:ff:ff:ff:ff	802.11	217	Beacon frame, SN=2961, FN=0, Flags=....., BI=100, SSID="AjouUniv_GUEST"
3	0.202332	fc:7f:f1:a2:d1:61	ff:ff:ff:ff:ff:ff	802.11	241	Beacon frame, SN=2962, FN=0, Flags=....., BI=100, SSID="AjouUniv_PRIVATE"
4	0.303560	fc:7f:f1:a2:d1:62	ff:ff:ff:ff:ff:ff	802.11	218	Beacon frame, SN=2963, FN=0, Flags=....., BI=100, SSID="AjouUniv_PUBLIC"
5	0.404765	fc:7f:f1:a2:d1:60	38:7a:0e:65:ab:ad	802.11	202	Probe Response, SN=2964, FN=0, Flags=....., BI=100, SSID="AjouUniv_GUEST"
6	0.505874	c8:b5:ad:70:76:00	ff:ff:ff:ff:ff:ff	802.11	241	Beacon frame, SN=799, FN=0, Flags=....., BI=100, SSID="AjouUniv_PRIVATE"
7	0.507765	58:86:94:01:dc:f8	ff:ff:ff:ff:ff:ff	802.11	297	Beacon frame, SN=447, FN=0, Flags=....., BI=100, SSID="gicooiptime24G"
8	0.507768	fc:7f:f1:a2:d1:60	b0:7d:64:85:29:1a	802.11	202	Probe Response, SN=2967, FN=0, Flags=....., BI=100, SSID="AjouUniv_GUEST"
9	0.507768	fc:7f:f1:a2:d1:61	b0:7d:64:85:29:1a	802.11	226	Probe Response, SN=2968, FN=0, Flags=....., BI=100, SSID="AjouUniv_PRIVATE"
10	0.507769	fc:7f:f1:a2:d1:62	b0:7d:64:85:29:1a	802.11	203	Probe Response, SN=2969, FN=0, Flags=....., BI=100, SSID="AjouUniv_PUBLIC"
11	0.507770	fc:7f:f1:a2:d1:62	ff:ff:ff:ff:ff:ff	802.11	34	Acknowledgement, Flags=.....

> Frame 6: 241 bytes on wire (1928 bits), 241 bytes captured (1928 bits)
> Radiotap Header v0, Length 24
> 802.11 radio information
v IEEE 802.11 Beacon frame, Flags:

- Type/Subtype: Beacon frame (0x0008)
 - > Frame Control Field: 0x8000
.000 0000 0000 0000 = Duration: 0 microseconds
 - > Receiver address: ff:ff:ff:ff:ff:ff
 - > Destination address: ff:ff:ff:ff:ff:ff
 - > Transmitter address: c8:b5:ad:70:76:00
 - > Source address: c8:b5:ad:70:76:00
 - > BSS Id: c8:b5:ad:70:76:00
.... 0000 = Fragment number: 0
0011 0001 1111 = Sequence number: 799
[WLAN Flags:
- v IEEE 802.11 Wireless Management
 - v Fixed parameters (12 bytes)
 - Timestamp: 1831945541544
 - Beacon Interval: 0.102400 [Seconds]
 - > Capabilities Information: 0x0431
 - v Tagged parameters (181 bytes)
 - v Tag: SSID parameter set: "AjouUniv_PRIVATE"
 - Tag Number: SSID parameter set (0)
 - Tag length: 16
 - SSID: "AjouUniv_PRIVATE"

0000 00 00 18 00 2e 40 00 a0 20 08 00 00 00 02 6c 09@... ..1.
0010 a0 00 af 00 00 00 af 00 80 00 00 00 ff ff ff ff
0020 ff ff c8 b5 ad 70 76 00 c8 b5 ad 70 76 00 f0 31pv...pv...1
0030 a8 97 76 88 aa 01 00 00 64 00 31 04 00 10 41 6a ...v.....d-1...Aj
0040 6f 75 55 6e 69 76 5f 50 52 49 56 41 54 45 01 08 ouUniv_P RIVATE..
0050 82 84 0b 16 24 30 48 6c 03 01 01 05 04 00 01 00\$0H1
0060 00 2a 01 06 32 04 0c 12 18 60 30 14 01 00 00 0f ...*.2... ^0.....
0070 ac 04 01 00 00 0f ac 04 01 00 00 0f ac 01 28 00(.....
0080 2d 1a ad 01 17 ff ff 00 00 00 00 00 00 00 00 00
0090 00 00 00 00 00 00 00 00 00 00 00 00 3d 16 01 00=.....
00a0 13 00 00 00 00 00 00 00 00 00 00 00 00 00 00J.....
00b0 00 00 00 00 4a 0e 14 00 0a 00 2c 01 c8 00 14 00
00c0 05 00 19 00 7f 08 05 00 08 00 00 00 00 40 dd 18@..
00d0 00 50 f2 02 01 01 80 00 03 a4 00 00 27 a4 00 00P.....
00e0 42 43 5e 00 62 32 2f 00 dd 07 00 0b 86 01 04 08 BC^b2/.
00f0 08

Tag (wlan.tag), 18바이트 패킷: 100 프로필: Default 1/50



RADIOTAP

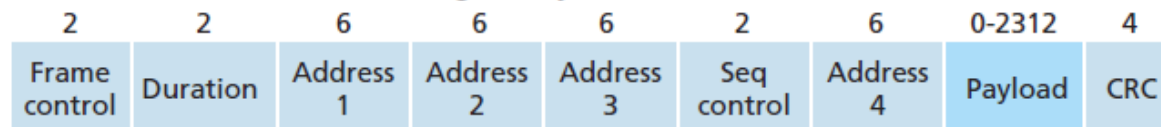
Programming IEEE 802.11 Header

❖ 802.11 frame header

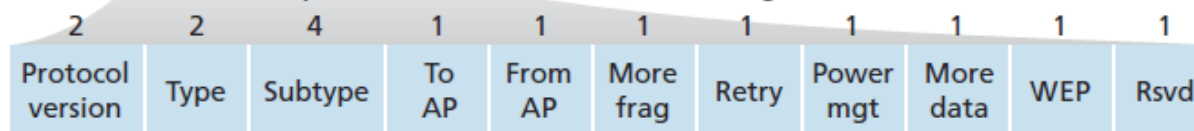
- /usr/include/linux/ieee80211.h

```
struct ieee80211_hdr {  
    uint16_t      frame_control;  
    uint16_t      duration_id;  
    uint8_t       addr1[6];  
    uint8_t       addr2[6];  
    uint8_t       addr3[6];  
    uint16_t      seq_ctrl;  
    // uint8_t     addr4[6];  
} __attribute__((packed));
```

Frame (numbers indicate field length in bytes):



Frame control field expanded (numbers indicate field length in bits):



Programming IEEE 802.11 Header

❖ 802.11 frame header

■ frame control

Frame control field expanded (numbers indicate field length in bits):

2	2	4	1	1	1	1	1	1	1	1
Protocol version	Type	Subtype	To AP	From AP	More frag	Retry	Power mgt	More data	WEP	Rsvd

```
#define WLAN_FC_TYPE_DATA      2
#define WLAN_FC_SUBTYPE_DATA   0

uint8_t fcchunk[2];          /* 802.11 header frame control */
fcchunk[0] = ((WLAN_FC_TYPE_DATA << 2) | (WLAN_FC_SUBTYPE_DATA << 4));
fcchunk[1] = 0x02;
memcpy(&hdr->frame_control, &fcchunk[0], 2*sizeof(uint8_t));
```

■ duration & others

```
hdr->duration_id = 0xffff;

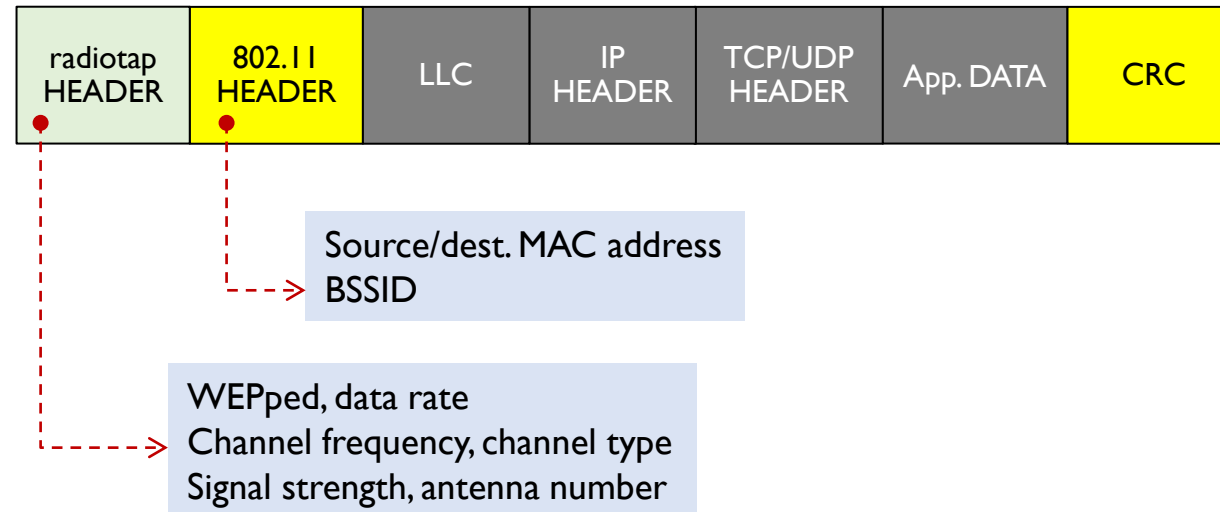
memcpy(&hdr->addr1[0], mac1, 6*sizeof(uint8_t));
memcpy(&hdr->addr2[0], mac2, 6*sizeof(uint8_t));
memcpy(&hdr->addr3[0], mac3, 6*sizeof(uint8_t));

hdr->seq_ctrl = 0;
```

Radiotap Header

❖ Radiotap

- de facto standard for 802.11 frame injection and reception.
 - found on its website: <http://www.radiotap.org/>
- more information about 802.11 frames
 - e.g. the signal strength, noise, whether a Frame Check Sequence is used to provide integrity, etc..





Radiotap Header

❖ Radiotap

■ Supported OSs

- FreeBSD
- Linux (Linux Wireless)
 - <http://linuxwireless.org/en/developers/Documentation/radiotap/>
- NetBSD
- OpenBSD
- Windows

Radiotap Header

❖ Radiotap header

- variable-length and extensible
- include/net/ieee80211_radiotap.h
 - base header (8 bytes)

```
struct ieee80211_radiotap_header {  
    u_int8_t      it_version;    // radiotap version, always 0  
    u_int8_t      it_pad;        // padding (or alignment)  
    u_int16_t     it_len;        // overall radiotap header length in bytes  
    u_int32_t     it_present;    // A bitmap telling which fields are present  
} __attribute__((__packed__));
```

▪ Structure of the header

```
< 8-byte ieee80211_radiotap_header (base header) >  
  
[ <possible argument bitmap extensions ... > ]  
  
[ <argument> ... ]
```

Radiotap Header

❖ Radiotap header

▪ Radiotap type for bitmap

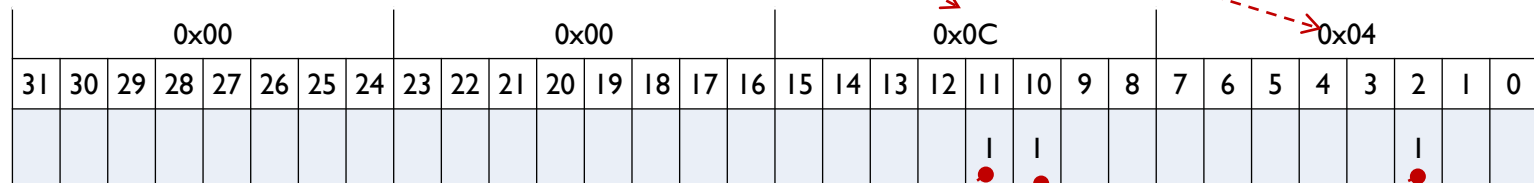
```
enum ieee80211_radiotap_type {  
    IEEE80211_RADIOTAP_TSFT          = 0,  
    IEEE80211_RADIOTAP_FLAGS         = 1,  
    IEEE80211_RADIOTAP_RATE          = 2,  
    IEEE80211_RADIOTAP_CHANNEL        = 3,  
    IEEE80211_RADIOTAP_FHSS           = 4,  
    IEEE80211_RADIOTAP_DBM_ANTSIGNAL  = 5,  
    IEEE80211_RADIOTAP_DBM_ANTNOISE   = 6,  
    IEEE80211_RADIOTAP_LOCK_QUALITY   = 7,  
    IEEE80211_RADIOTAP_TX_ATTENUATION = 8,  
    IEEE80211_RADIOTAP_DB_TX_ATTENUATION = 9,  
    IEEE80211_RADIOTAP_DBM_TX_POWER   = 10,  
    IEEE80211_RADIOTAP_ANTENNA        = 11,  
    IEEE80211_RADIOTAP_DB_ANTSIGNAL    = 12,  
    IEEE80211_RADIOTAP_DB_ANTNOISE     = 13,  
    IEEE80211_RADIOTAP_RX_FLAGS       = 14,  
    IEEE80211_RADIOTAP_TX_FLAGS       = 15,  
    IEEE80211_RADIOTAP_RTS_RETRIES     = 16,  
    IEEE80211_RADIOTAP_DATA_RETRIES    = 17,  
    IEEE80211_RADIOTAP_EXT             = 31  
};
```

Radiotap Header

❖ Radiotap header

■ example

```
0x00, 0x00,           // <-- radiotap version + pad byte
0x0b, 0x00,           // <- radiotap header length
0x04, 0x0c, 0x00, 0x00, // <-- bitmap
0x6c,                 // <-- rate (in 500kHz units)
0x0c,                 // <-- tx power
0x01                  // <-- antenna
```



IEEE80211_RADIOTAP_RATE = 2,
IEEE80211_RADIOTAP_DBM_TX_POWER = 10
IEEE80211_RADIOTAP_ANTENNA = 11

Radiotap Header

- ❖ Radiotap header
 - Example capture

Source -
<https://mrncciew.com/2014/10/17/cwap-spectrum-analysis/>

```
Frame 1: 276 bytes on wire (2208 bits), 276 bytes captured (2208 bits) on interf
Radiotap Header v0, Length 18
Header revision: 0
Header pad: 0
Header length: 18
Present flags
...0 = TSFT: False
...1 = Flags: True
...1 = Rate: True
...1 = Channel: True
...0 = FHSS: False
...1 = dBm Antenna Signal: True
...0 = dBm Antenna Noise: False
...0 = Lock Quality: False
...0 = TX Attenuation: False
...0 = dB TX Attenuation: False
...0 = dBm TX Power: False
...1 = Antenna: True
...0 = dB Antenna Signal: False
...0 = dB Antenna Noise: False
...1 = RX flags: True
...0 = Channel+: False
...0 = HT information: False
...0 = A-MPDU Status: False
...0 = VHT information: False
...0 0000 00 = Reserved: 0x00000000
...0 = Radiotap NS next: False
...0 = Vendor NS next: False
...0 = Ext: False
Flags: 0x10
...0 = CFP: False
...0 = Preamble: Long
...0 = WEP: False
...0 = Fragmentation: False
...1 = FCS at end: True
...0 = Data Pad: False
...0 = Bad FCS: False
...0 = Short GI: False
Data Rate: 24.0 Mb/s
Channel frequency: 5180 [A 36]
Channel type: 802.11a (0x0140)
...0 = Turbo: False
...0 = Complementary Code Keying (CCK): False
...1 = Orthogonal Frequency-Division Multiplexing (OFDM): True
...0 = 2 GHz spectrum: False
...1 = 5 GHz spectrum: True
...0 = Passive: False
...0 = Dynamic CCK-OFDM: False
...0 = Gaussian Frequency Shift Keying (GFSK): False
...0 = GSM (900MHz): False
...0 = Static Turbo: False
...0 = Half Rate Channel (10MHz Channel Width): False
...0 = Quarter Rate Channel (5MHz Channel Width): False
SSI Signal: -29 dBm
Antenna: 5
RX Flags: 0x0000
...0 = Bad PLCP: False
IEEE 802.11 Beacon frame, Flags: .....C
IEEE 802.11 wireless LAN management frame
```

Example – wlan_pk_gen_ex.c (1/5)

```
int main()
{
    pcap_if_t          *alldevs;
    pcap_if_t          *d;
    int                 i, dnum, snum;

    pcap_findalldevs(&alldevs, errbuf);    // Retrieve the device list

    ifprint0(alldevs, &dnum);              // Scan the list printing every entry

    // Select the interface for handling packets
    printf("Enter the interface number (1-%d):", dnum);
    scanf("%d", &snum);

    for (d = alldevs, i = 0; i < snum - 1; d = d->next, i++);

    adhandle = pcap_open_live(d->name, 65536, 1, 1000, effbuf); // Open the adapter

    generate_packet_wlan_udp(packet, &pklen);                  // UDP segment generation

    pcap_sendpacket(adhandle, packet, pklen);                  // Send packet
}
```

Example – wlan_pk_gen_ex.c (2/5)

```
void generate_packet_wlan_udp(u_char* packet, int *pklen)
{
    char                sendbuf[4096] = { 0 };
    uint8_t             *radioh;      // radiotap header
    struct ieee80211_hdr *wlanh;      // 802.11 header
    uint8_t             fcchunk[2];  // frame control in 802.11 header
    uint8_t             *llc;        // LLC part
    struct ip_header     *iph;        // IP header
    struct udp_header    *udph;      // TCP header
    char                *data;        // Application Data

    // Total buffer size (note the 0 bytes of data and the 4 bytes of FCS)
    *pklen = sizeof(u8aRadiotapHeader) + sizeof(struct ieee80211_hdr) + sizeof(ipllc)
            + sizeof(struct ip_header) + sizeof(struct udp_header)
            + strlen(app_data) + 4; // 4 bytes for FCS

    // Put our pointers in the right place
    radioh      = (uint8_t *) sendbuf;
    wlanh       = (struct ieee80211_hdr *) (radioh+sizeof(u8aRadiotapHeader));
    llc         = (uint8_t *) (wlanh+1);
    iph         = (struct ip_header *) (llc+sizeof(ipllc));
    udph        = (struct udp_header *) (iph+1);
    data        = (uint8_t *) (udph+1);
```

Example – wlan_pk_gen_ex.c (3/5)

```
// The radiotap header has been explained already
memcpy(radiah, u8aRadiotapHeader, sizeof(u8aRadiotapHeader));

// construct the 802.11 header
fcchunk[0] = ((WLAN_FC_TYPE_DATA << 2) | (WLAN_FC_SUBTYPE_DATA << 4));
fcchunk[1] = 0x02;
memcpy(&wlanh->frame_control, &fcchunk[0], 2*sizeof(uint8_t));
wlanh->duration_id = 0xffff;
memcpy(&wlanh->addr1[0], src_mac, 6*sizeof(uint8_t));
memcpy(&wlanh->addr2[0], dst_mac, 6*sizeof(uint8_t));
memcpy(&wlanh->addr3[0], ssid_mac, 6*sizeof(uint8_t));
wlanh->seq_ctrl = 0;
//wlanh->addr4;

// The LLC+SNAP header has already been explained above
memcpy(llc, ipllc, 8*sizeof(uint8_t));

// make IP, UDP, and data part
get_header_ip(iph);           // IP header
get_header_udp(udph);         // UDP header
strcpy(data, app_data);       // Application data

memcpy(packet, sendbuf, sizeof(sendbuf));
}
```

Example – wlan_pk_gen_ex.c (4/5)

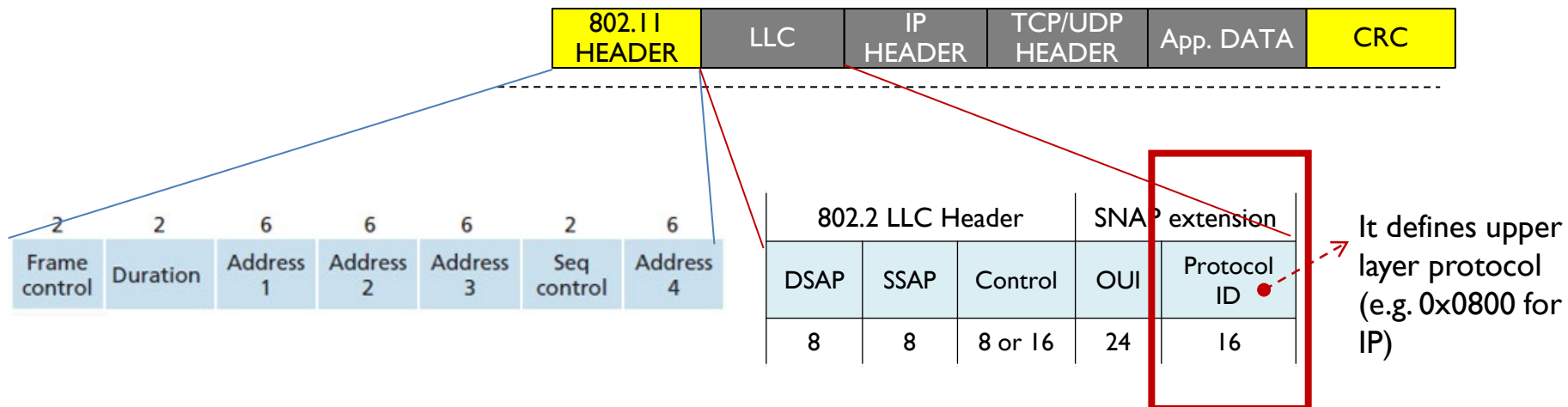
```
// Radiotap Header
static const uint8_t u8aRadiotapHeader[] = {
    0x00,                // u_int8_t  it_version : major version of the radiotap header
    0x00,                // u_int8_t  it_pad : unused
    0x18, 0x00,          // u_int16_t it_len : entire length of the radiotap (in bytes)
    0x0f, 0x80, 0x00, 0x00, // u_int32_t it_present : a bitmask of the radiotap data field
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // timestamp
    0x10,                // flags: IEEE80211_RADIOTAP_F_FCS, the card adds FCS
    0x00,                // rate (in 500KHz unit)
    0x00, 0x00, 0x00, 0x00, // channel
    0x08, 0x00,          // IEEE80211_RADIOTAP_F_TX_NOACK, the card won't wait for an ACK
};

// 802.11 wlan header, defined in include/linux/ieee80211.h
struct ieee80211_hdr {
    uint16_t    frame_control;
    uint16_t    duration_id;
    uint8_t     addr1[6];
    uint8_t     addr2[6];
    uint8_t     addr3[6];
    uint16_t    seq_ctrl;
    //uint8_t     addr4[6];
} __attribute__((packed));
```

Example – wlan_pk_gen_ex.c (5/5)

```
#define WLAN_FC_TYPE_DATA 2
#define WLAN_FC_SUBTYPE_DATA 0

// logical link control (LLC) header should
const uint8_t ipllc[8] = { 0xaa, 0xaa, 0x03, 0x00, 0x00, 0x00, 0x08, 0x00 };
```



Example – wlan_pk_gen_ex.c

❖ Run

Available devices:

1. ens33 - No description available 192.168.247.128
2. any - Pseudo-device that captures on all interfaces
3. lo - No description available 127.0.0.1
4. wlan0 - No description available
5. bluetooth-monitor - Bluetooth Linux Monitor
6. nflog - Linux netfilter log (NFLOG) interface
7. nfqueue - Linux netfilter queue (NFQUEUE) interface
8. dbus-system - D-Bus system bus
9. dbus-session - D-Bus session bus

Enter the interface number (1-9):4

in gen_udp(): psheln=50 d[0]=H d[1]= e checksum1 = 57175, checksum2 = 0

Example – wlan_pk_gen_ex.c

❖ Run

The image shows a Wireshark packet capture analysis of a UDP packet. The packet list at the top shows two packets, both from 169.254.1.1 to 255.255.255.255 on port 9999. The selected packet (No. 2) is expanded to show the IEEE 802.11 Data and Internet Protocol Version 4 details.

Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	169.254.1.1	255.255.255.255	UDP	118	9999 → 80 Len=30
2	0.000995	169.254.1.1	255.255.255.255	UDP	101	9999 → 80 Len=30

Packet Details:

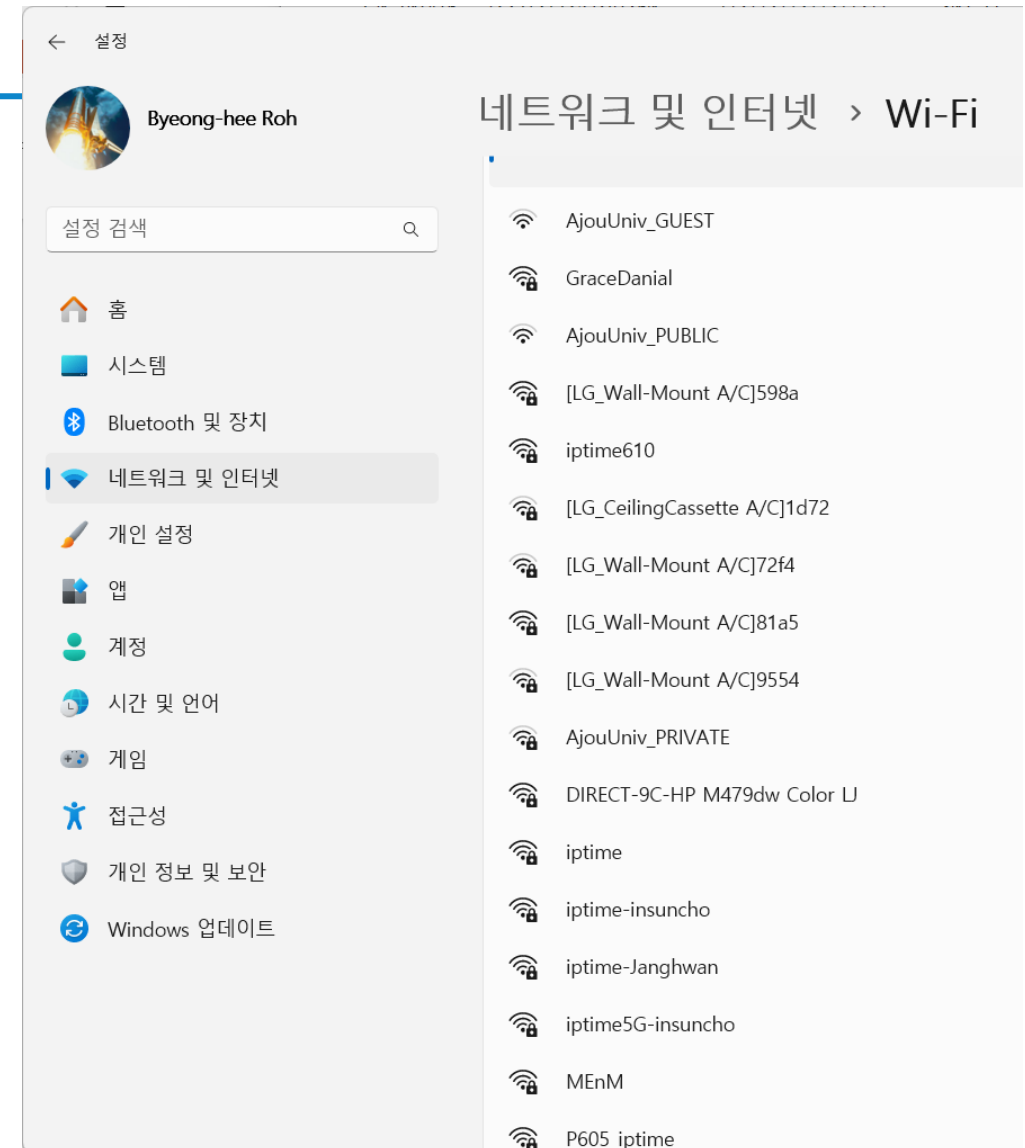
- 802.11 radio information
 - Data rate: 0.0 Mb/s
 - TSF timestamp: 0
- IEEE 802.11 Data, Flags:F.C
 - Type/Subtype: Data (0x0020)
 - Frame Control Field: 0x0802
 -00 = Version: 0
 - 10.. = Type: Data frame (2)
 - 0000 = Subtype: 0
 - Flags: 0x02
 -10 = DS status: Frame from DS to a STA via AP(To DS: 0 From DS: 1) (0x2)
 -0.. = More Fragments: This is the last fragment
 - 0... = Retry: Frame is not being retransmitted
 - ...0 = PWR MGT: STA will stay up
 - ..0. = More Data: No data buffered
 - .0.. = Protected flag: Data is not protected
 - 0... = +HTC/Order flag: Not strictly ordered
 - Duration/ID: 16383 (reserved)
 - Receiver address: 01:23:45:67:89:ab
 - Transmitter address: ff:ff:ff:ff:ff:ff
 - Destination address: 01:23:45:67:89:ab
 - Source address: b1:b2:b3:b4:b5:b6
 - BSS Id: ff:ff:ff:ff:ff:ff
 - STA address: 01:23:45:67:89:ab
 - 0000 = Fragment number: 0
 - 0000 0000 0000 = Sequence number: 0
 - Frame check sequence: 0x00000000 [unverified]
 - [FCS Status: Unverified]
 - [WLAN Flags:F.C]
- Logical-Link Control
 - DSAP: SNAP (0xaa)
 - SSAP: SNAP (0xaa)
 - Control field: U, func=UI (0x03)
 - Organization Code: 00:00:00 (Officially Xerox, but
 - Type: IPv4 (0x0800)
- Internet Protocol Version 4, Src: 169.254.1.1, Dst: 255.255.255.255
 - 0100 = Version: 4
 - 0101 = Header Length: 20 bytes (5)

Packet Bytes:

```
00 00 18 00 0f 80 00 00 00 00 00 00 00 00 00 00 .....#t
10 00 00 00 00 00 08 00 08 02 ff ff 01 23 45 67 .....#t
89 ab ff ff ff ff ff ff b1 b2 b3 b4 b5 b6 00 00 .....#t
aa aa 03 00 00 00 08 00 45 00 00 3a d4 31 40 00 .....E...10
40 11 bb 82 a9 fe 01 01 ff ff ff ff 27 0f 00 50 @.....'
00 26 57 df 48 65 6c 6c 6f 20 57 6f 72 6c 64 2c .&W.Hell o World
20 49 20 61 6d 20 74 68 65 20 48 65 72 6f 20 21 I am th e Hero
21 21 00 00 00 00 !!....
```


Suggested Individual Task

- ❖ Write a program
 - to list all SSIDs (WLAN names) that found through monitor mode



Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN



Mobile **Multimedia Convergence** Network Lab.

Homepage: <http://mmcn.ajou.ac.kr>

facebook: <http://www.facebook.com/#!/groups/190702010947314>

